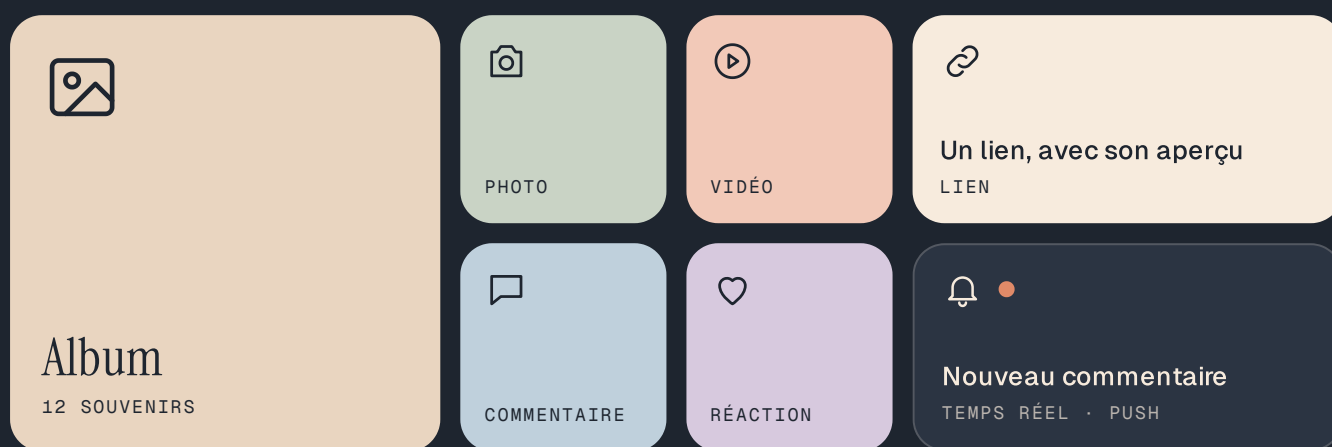


moments.

Journal de souvenirs partagés

Une app web privée où famille et proches rangent leurs souvenirs dans des espaces communs — albums, photos, vidéos, liens — les commentent, y réagissent, et sont prévenus en direct quand quelqu'un ajoute quelque chose.



5

espaces partagés, chacun avec ses albums et ses membres

~7

utilisateurs — famille et proches, sur téléphone

15

thèmes visuels, pilotés par des tokens CSS

1

concepteur-développeur, de la spécification au déploiement

Des souvenirs à soi, *partagés à quelques-uns.*

moments. répond à un besoin simple : un endroit commun pour la famille et les proches, hors des réseaux sociaux. Le prototype a d'abord été généré sur une plateforme de développement par IA, puis reconstruit sur une stack maîtrisée pour gagner en autonomie et en fiabilité.

Cercle privé	Chaque espace n'est visible que de ses membres. L'accès est contrôlé par les règles de sécurité de la base, pas par l'interface.
Aucun environnement de test	Tout part directement en production. Chaque changement doit donc être petit, ciblé et réversible.
Socle partagé	Le même compte cloud héberge un autre projet, Keepr. Un déploiement de l'un ne doit jamais écraser la configuration de l'autre.
Mobile d'abord	L'app vit surtout sur téléphone, installée en PWA : les notifications doivent fonctionner jusque sur iPhone.
Autonomie	Objectif de long terme : ne dépendre d'aucune plateforme tierce pour faire vivre l'app.

RÉPONDRE À

Où ranger ?

Des espaces, des albums, et des souvenirs de toute nature : photo, vidéo, lien.

RÉPONDRE À

Comment échanger ?

Des commentaires et des réactions sur chaque souvenir, visibles par l'espace.

RÉPONDRE À

Comment être prévenu ?

Une cloche en temps réel dans l'app et des notifications push sur l'appareil.

CE QUE MOMENTS. N'EST PAS

- Un réseau social public
- Un fil trié par algorithme
- Une vitrine : rien n'est indexé ni partagé dehors
- Un compte de plus à créer : connexion Google

Reconstruire *sans rien casser.*

En juin 2026, l'app quitte sa stack d'origine — générée et hébergée par une plateforme de développement par IA — pour Firebase. Les utilisateurs gardent leurs souvenirs, leurs comptes et leurs habitudes.

AVANT

Prototype généré

IDE IA Emergent

Backend FastAPI

Base MongoDB

Hébergement de la plateforme



APRÈS

Stack maîtrisée

Firestore · base temps réel

Storage · photos et vidéos

Auth · connexion Google

Cloud Functions v2 · région Europe

Hosting · PWA installable

LE DÉTAIL QUI A TOUT SIMPLIFIÉ

Les identifiants de connexion Firebase reprennent **exactement** les anciens identifiants utilisateurs. Toutes les données existantes restaient rattachées à la bonne personne : aucun composant consommateur n'a eu à être réécrit.

MODÈLE DE DONNÉES

spaces/{slug}

Le contenu partagé d'un espace

albums/

items/

comments/

reactions/

users/{uid}

L'état personnel de chaque membre

notifications/

sa boîte, lue en temps réel

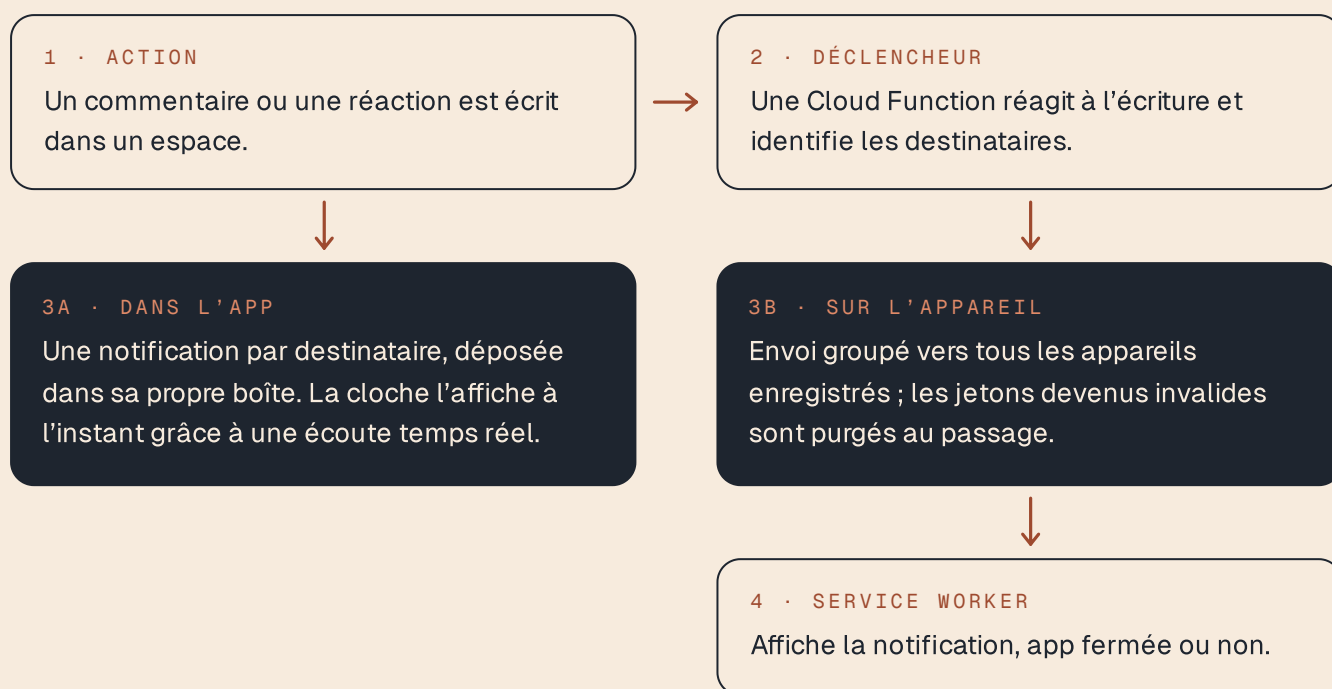
fcmTokens/

ses appareils joignables en push

Le contenu vit dans les espaces ; tout ce qui est propre à une personne vit sous son identifiant. Une séparation qui simplifie à la fois les règles d'accès et la lecture temps réel.

Être prévenu, *sans rien guetter.*

Quand un proche commente ou réagit, chaque membre concerné le voit en direct dans l'app — et reçoit une notification sur son téléphone, iPhone compris.



UN PARCOURS PAR APPAREIL

iPhone, Safari, ordinateur

L'activation des notifications détecte la situation — app installée sur iPhone, Safari en onglet, navigateur de bureau — et adapte le parcours. Push confirmé fonctionnel sur iOS.

VALIDÉ AVANT D'ÊTRE CODÉ

Le panneau de notifications

Transparence, superpositions et débordement mobile : corrigés sur maquette interactive d'abord — fenêtre opaque sur ordinateur, panneau glissant depuis le bas sur téléphone.

Chaque choix *a un prix, assumé.*

Les arbitrages sont documentés comme des décisions d'architecture : l'option retenue, celle qui a été écartée, et ce que l'on accepte de perdre en échange.

DÉCISION	ALTERNATIVE ÉCARTÉE	RAISON	COÛT ACCEPTÉ
Migrer vers Firebase	Garder la stack d'origine	Temps réel, auth, médias et push intégrés ; plus de serveur à maintenir.	Dépendance à un cloud : une étape vers l'autonomie, pas l'arrivée.
Réutiliser les identifiants existants	Remapper tous les identifiants	Aucun composant consommateur réécrit pendant la migration.	Un couplage durable à l'historique des anciens identifiants.
Notifications distribuées côté serveur	Calcul à la lecture, côté client	Une boîte par personne, lue en temps réel sans requête complexe.	Une écriture par destinataire à chaque événement.
Push natif de la plateforme	Service de push tiers	Mêmes identités, même facturation ; purge automatique des appareils morts.	Sur iPhone, le push exige l'app installée en PWA.
Règles de sécurité hors du dépôt de l'app	Règles versionnées avec l'app	Un déploiement de moments. n'écrase jamais celles de Keepr.	Deux référentiels à garder cohérents.
Déploiement par surface	Un déploiement global	Sans staging : rayon d'impact réduit, retour arrière ciblé.	Plus de commandes, plus de discipline.
Thèmes portés par des tokens CSS	Une feuille de style par thème	15 thèmes sans dupliquer un seul composant.	Toute nouvelle couleur doit passer par un token.
Vidéos iOS téléchargées vers Fichiers	Conversion serveur vers MP4	Aucun pipeline de transcodage à opérer.	Pas d'enregistrement direct dans Photos sur iPhone.

PRINCIPE DIRECTEUR

Sans environnement de test, chaque mise en production doit être *petite, ciblée et réversible.*

Livrer seul, *avec un agent IA cadré.*

Le chat (Claude) conçoit, spécifie et rédige les consignes ; l'agent (Claude Code) exécute. Le concepteur n'écrit pas le code : il cadre, valide chaque étape et exige des preuves.

Maquette avant code	Chaque parcours est validé sur une maquette HTML interactive avant toute implémentation.
Une étape, une validation	Séquence explicite ; rien ne passe à l'étape suivante sans vérification.
Coût estimé d'abord	Chaque tâche d'agent est estimée avant d'être lancée.
Règles diffées	Les règles de sécurité sont comparées à la version déployée avant toute modification.
Déploiement par surface	Fonctions, règles et hébergement partent séparément, avec des commandes explicites.

PIÈGES CONSIGNÉS

Refus d'accès silencieux

Une écoute temps réel refusée échoue sans erreur visible → toujours vérifier les règles réellement déployées.

Champs indéfinis

La base les rejette, contrairement à une API REST → nettoyage systématique avant chaque écriture.

Éditions partielles

Imports orphelins et code résiduel → les fichiers modifiés sont remplacés en entier.

Contexte périmé

La doc de l'agent décrivait encore l'ancienne stack → toute incohérence est signalée explicitement.

STACK

React · Tailwind CSS
PWA installable
Firestore · Storage · Auth
Cloud Functions v2 · FCM
Firebase Hosting
Claude · Claude Code

CHANTIERS OUVERTS

Notes vocales, jusqu'à 5 min
Fonctions en Node 22
Aperçus de liens enrichis
Nettoyage des médias orphelins
Invitations expirant à 7 jours

CONCEPTION & RÉALISATION

Mathieu El Almawie

Business Analyst MOA · PO
conception produit et architecture
de moments.

mthlabs.github.io/Portofolio
mathieu.elalmawie@gmail.com